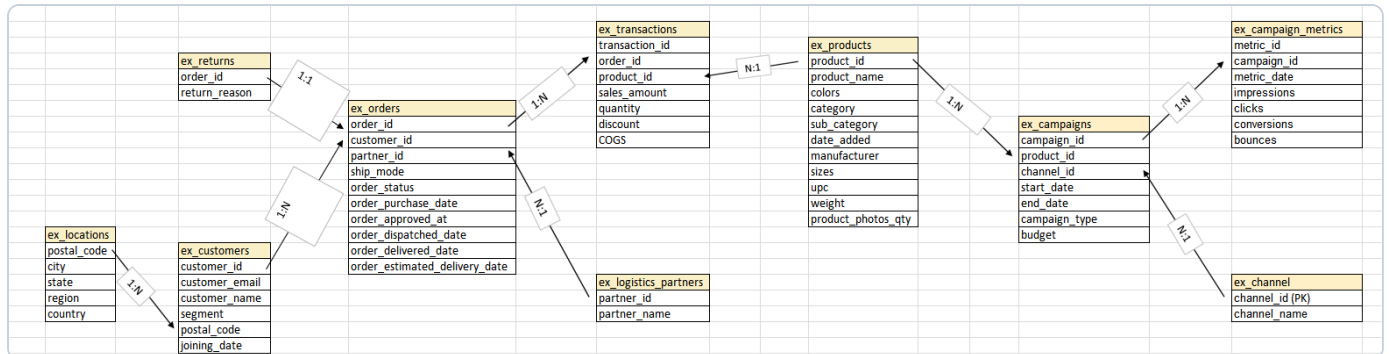


Day 3 Individual SQL Assessment — Answer Key

Same 3 problems as the student booklet (generic form below — students see their own name woven into the problem text instead of "the learner"). Reference solution and a grading rubric follow each. Column casing matches the real ER diagram exactly (lowercase, except COGS).



Q1 The Logistics Partner Scorecard

For each logistics partner: delivered order count, average days purchase-to-delivery, ranked fastest to slowest.
Output: partner_name | delivered_order_count | avg_delivery_days | speed_rank.

Reference solution

```

WITH partner_perf AS (
  SELECT
    lp.partner_name,
    COUNT(o.order_id) AS delivered_order_count,
    AVG(DATEDIFF(day, o.order_purchase_date, o.order_delivered_date)) AS avg_delivery_days
  FROM ex_orders o
  JOIN ex_logistics_partners lp ON o.partner_id = lp.partner_id
  WHERE o.order_status = 'delivered'
    AND o.order_delivered_date IS NOT NULL
  GROUP BY lp.partner_name
)
SELECT
  partner_name,
  delivered_order_count,
  avg_delivery_days,
  RANK() OVER (ORDER BY avg_delivery_days ASC) AS speed_rank
FROM partner_perf
ORDER BY speed_rank;
  
```

Rubric

- Filters to order_status = 'delivered' (and/or delivered_date IS NOT NULL) BEFORE aggregating — otherwise the average is polluted by orders with no delivery date
- DATEDIFF(day, purchase, delivered) wrapped in AVG(), grouped by partner
- RANK() ordered ASCENDING on avg_delivery_days — flag this explicitly if a learner defaults to DESC out of habit from earlier ranking exercises
- Accept ROW_NUMBER() or DENSE_RANK() in place of RANK() — same intent

Q2 Profit by Segment, Year over Year

Total profit (sales_amount - COGS) per segment, 2017 vs 2018 side by side. Output: segment | [2017] | [2018].

Reference solution

```
WITH segment_profit AS (  
  SELECT  
    c.segment,  
    YEAR(o.order_purchase_date) AS order_year,  
    SUM(t.sales_amount - t.COGS) AS profit  
  FROM ex_transactions t  
  JOIN ex_orders o ON t.order_id = o.order_id  
  JOIN ex_customers c ON o.customer_id = c.customer_id  
  WHERE YEAR(o.order_purchase_date) IN (2017, 2018)  
  GROUP BY c.segment, YEAR(o.order_purchase_date)  
)  
SELECT segment, [2017], [2018]  
FROM segment_profit  
PIVOT(  
  SUM(profit)  
  FOR order_year IN ([2017],[2018])  
) AS pvt  
ORDER BY segment;
```

Rubric

- Long-format aggregation built first: segment × year × profit
- Profit is computed as SUM(sales_amount - COGS), not SUM(sales_amount) - SUM(COGS) done separately then subtracted outside the aggregate — both are mathematically equivalent here, accept either
- PIVOT block structured correctly: aggregate / FOR order_year / IN ([2017],[2018])
- Accept an equivalent MAX(CASE WHEN order_year = 2017 THEN profit END) rewrite in place of PIVOT

Q3 Spot the Slow Query

Learner is given this query and asked to name 3 specific issues, then rewrite it.

```
SELECT DISTINCT *
FROM ex_customers c, ex_orders o, ex_transactions t
WHERE c.customer_id = o.customer_id
      AND o.order_id = t.order_id
GROUP BY c.segment, o.order_status
HAVING c.segment = 'Consumer' AND o.order_status = 'delivered'
```

The 3 required findings

- **SELECT DISTINCT *** — fetches every column unnecessarily, and pairs DISTINCT with a GROUP BY that already dedupes; pick one, and neither should be a blanket *.
- **Filtering in HAVING instead of WHERE** — `c.segment = 'Consumer'` and `o.order_status = 'delivered'` are row-level filters, not aggregate conditions. They belong in WHERE, before the GROUP BY, not after it.
- **Wrong join order / old comma-join syntax** — starts FROM the smallest table (customers) and comma-joins outward instead of starting FROM the highest-granularity table (transactions) with explicit JOIN syntax.

Reference rewrite

```
SELECT
  c.segment,
  o.order_status,
  COUNT(DISTINCT o.order_id) AS order_count
FROM ex_transactions t
JOIN ex_orders o      ON t.order_id = o.order_id
JOIN ex_customers c  ON o.customer_id = c.customer_id
WHERE c.segment = 'Consumer'
      AND o.order_status = 'delivered'
GROUP BY c.segment, o.order_status;
```

Rubric

- Full marks requires naming all 3 issues above (in their own words is fine)
- Rewrite must use WHERE for the two filters, explicit JOIN syntax starting FROM ex_transactions, and select only needed columns
- COUNT(DISTINCT order_id) in the rewrite is correct and NOT the same mistake as SELECT DISTINCT * — don't penalize it